CMSC 430 Project 1 Documentation
Student: Joe Merrill
Date: 18 March 2026

1. Approach

I began by extracting the project requirements from the provided PDF files and comparing them to the supplied skeleton scanner, token header, and listing module. The original skeleton supported only the smaller Summer 2023 token set, so the first step was to identify the new lexical categories and compilation-listing requirements introduced in Project 1.

The scanner was then extended to support the added reserved words, operators, hexadecimal integer literals, real literals, and expanded character literal escapes. Identifier handling was updated to support underscores while enforcing the assignment constraints: no leading underscore, no trailing underscore, and no more than two consecutive underscores. Comment handling was also expanded to support both // and -- line comments.

The compilation listing module was modified separately from the scanner so the error-reporting behavior stayed centralized. Instead of remembering only one error message at a time, it now queues every error generated on a line and prints them all before advancing to the next line. The footer was changed so successful runs print Compiled Successfully, while failing runs print separate totals for lexical, syntax, and semantic errors.

Because Flex was not installed in the local workspace, I kept two compatible scanner representations in the submission folder. The required deliverable file, scanner.l, reflects the assignment rules. A standalone scanner.cpp was also kept so the project could still be built and verified in the current environment.

2. Test Plan

The following supplied tests were used as the primary verification set.

test1.txt

Purpose: verifies a simple valid program with arithmetic expressions and no lexical errors.
Expected result: no lexical errors, Compiled Successfully.

```
PS C:\Users\jamer\Desktop\CMSC 430 Compiler Theory and Design\Project 1\Project 1 Skeleton Code> cmd /c "type ""..\Project 1 Test Data\test1.
txt"" | compile.exe"

    1  // Function with Arithmetic Expression
    2
    3  function main returns integer;
    4  begin
    5      7 + 2 * (5  + 4);
    6  end;
    7
Compiled Successfully
```

test2.txt

Purpose: verifies that the scanner reports a single lexical error when an invalid

character appears in otherwise valid input.
Expected result: one lexical error, zero syntax errors, zero semantic errors.
Observed output summary: the scanner reports the invalid $ character on the affected source line.

```
PS C:\Users\jamer\Desktop\CMSC 430 Compiler Theory and Design\Project 1\Project 1 Skeleton Code> cmd /c "type ""..\Project 1 Test Data\test2.
txt"" | .\compile.exe"

   1  // Function with a lexical error
   2
   3  function main returns integer;
   4  begin
   5      7 * 2 $ (2  + 4);
Lexical Error, Invalid Character $
   6  end;
   7
Lexical Errors 1
Syntax Errors 0
Semantic Errors 0
```

test3.txt
Purpose: verifies punctuation symbols, identifiers, literals, operators, and the original reserved words in a compact token smoke test.
Expected result: no lexical errors, Compiled Successfully.
Observed output summary: the scanner accepts each baseline lexeme category without reporting errors.

```
PS C:\Users\jamer\Desktop\CMSC 430 Compiler Theory and Design\Project 1\Project 1 Skeleton Code> cmd /c "type ""..\Project 1 Test Data\test3.
txt"" | .\compile.exe"

   1  // Punctuation symbols
   2
   3  ,;() =>
   4
   5  // Identifier
   6
   7  name name123
   8
   9  // Literals
  10
  11  123 'a'
  12
  13  // Logical operator
  14
  15  &
  16
  17  // Relational operator
  18
  19  <
  20
  21  // Arithmetic operators
  22
  23  + *
  24
  25  // Reserved words
  26
  27  begin case character end endswitch function is integer list of returns switch when
  28
Compiled Successfully
```

test4.txt
Purpose: verifies the additional reserved words such as else, elsif, endif, endfold, fold, if, left, real, right, and then.
Expected result: no lexical errors, Compiled Successfully.

```
PS C:\Users\jamer\Desktop\CMSC 430 Compiler Theory and Design\Project 1\Project 1 Skeleton Code> cmd /c "type ""..\Project 1 Test Data\test4.
txt"" | .\compile.exe"

  1  // Function with All Reserved Words
  2
  3  function main returns character;
  4      number: real is when 2 < 3, 0 : 1;
  5      values: list of integer is (4, 5, 6);
  6  begin
  7      if number < 6.3 then
  8          fold left + (1, 2, 3) endfold;
  9      elsif 6 < 7 then
 10          fold right + values endfold;
 11      else
 12          switch a is
 13              case 1 => number + 2;
 14              case 2 => number * 3;
 15              others => number;
 16          endswitch;
 17      endif;
 18  end;
 19
Compiled Successfully
```

test5.txt

Purpose: verifies the added logical, relational, and arithmetic operators including |, !, =, <>, >=, <=, -, /, %, ^, and ~.

Expected result: no lexical errors, Compiled Successfully.

```
PS C:\Users\jamer\Desktop\CMSC 430 Compiler Theory and Design\Project 1\Project 1 Skeleton Code> cmd /c "type ""..\Project 1 Test Data\test5.
txt"" | .\compile.exe"

  1  // Program Containing the New Operators
  2
  3  function main b: integer, c: integer returns integer;
  4       a: integer is 3;
  5  begin
  6      if (a < 2) | (a > 0) & (~b <> 0) then
  7          7 - 2 / (9 % 4);
  8      else
  9          if b >= 2 | b <= 6 & !(c = 1) then
 10              7 + 2 * (2 + 4);
 11          else
 12              a ^ 2;
 13          endif;
 14      endif;
 15  end;
 16
Compiled Successfully
```

test6.txt

Purpose: verifies -- comments, modified identifiers, real literals, hexadecimal integer literals, and escaped character literals.

Expected result: no lexical errors, Compiled Successfully.

```
PS C:\Users\jamer\Desktop\CMSC 430 Compiler Theory and Design\Project 1\Project 1 Skeleton Code> cmd /c "type ""..\Project 1 Test Data\test6.
txt"" | .\compile.exe"

  1  // Program Containing the New Comment, Modified Identifier
  2  //     and Real Literal and Hex and Character Literals
  3
  4  -- This is the new style comment
  5
  6  function main b: integer, c: integer returns integer;
  7       a: real is .3;
  8       d: real is 5.7;
  9       a__1: real is .4e2;
 10       ab_c_d: real is 4.3E+1;
 11       ab1_cd2: real is 4.5e-1;
 12       hex: integer is #2aF;
 13       char1: character is 'C';
 14       char2: character is '\n';
 15  begin
 16       hex + 2;
 17  end;
 18
Compiled Successfully
```

test7.txt

Purpose: verifies that multiple lexical errors on the same line are all reported and counted correctly.
Expected result: two lexical errors, zero syntax errors, zero semantic errors.
Observed output summary: the scanner reported both invalid characters ($ and ?) on the same source line.

```
PS C:\Users\jamer\Desktop\CMSC 430 Compiler Theory and Design\Project 1\Project 1 Skeleton Code> cmd /c "type ""..\Project 1 Test Data\test7.
txt"" | .\compile.exe"

   1  // Function with Two Lexical Errors
   2
   3  function main returns integer;
   4  begin
   5      7 $ 2 ? (2 + 4);
Lexical Error, Invalid Character $
Lexical Error, Invalid Character ?
   6  end;
   7
Lexical Errors 2
Syntax Errors 0
Semantic Errors 0
```

test8.txt

Purpose: verifies every punctuation symbol, operator, reserved word, valid identifier form, and invalid identifier form listed in the specification.
Expected result: five lexical errors from invalid underscores and lexeme-token output matching the supplied reference file.
Observed output summary: lexical error count matched the specification and generated lexemes.txt matched the provided reference.

```
PS C:\Users\jamer\Desktop\CMSC 430 Compiler Theory and Design\Project 1\Project 1 Skeleton Code> cmd /c "type ""..\Project 1 Test Data\test8.
txt"" | .\compile.exe"

   1  -- Punctuation symbols
   2
   3  ,:;() =>
   4
   5  // Valid identifiers
   6
   7  name_1
   8  name_1__a2_ab3
   9
  10  // Invalid identifiers
  11
  12  name___2
Lexical Error, Invalid Character _
Lexical Error, Invalid Character _
Lexical Error, Invalid Character _
  13  _name3
Lexical Error, Invalid Character _
  14  name4_
Lexical Error, Invalid Character _
  15
  16  // Integer Literals
  17
  18  23 #3aD
  18  23 #3aD
  19
  20  // Real Literals
  21
  22  123.45 .123 1.2E2 .1e+2 1.2E-2
  23
  24  // Character Literals
  25
  26  'A' '\n'
  27
  28  // Logical operators
  29
  30  & | !
  31
  32  // Relational operators
  33
  34  = <> > >= < <=
  35
  36  // Arithmetic operators
  37
  38  + - * / % ^ ~
  39
  40  // Reserved words
  41
  42  begin case character else elsif end endcase endfold endif endswitch
  43  fold function if integer is left list of others real returns right
  44  switch then when
  45
Lexical Errors 5
Syntax Errors 0
Semantic Errors 0
```

test9.txt
Purpose: custom test case created to verify mixed valid and invalid literal handling in one program, including a valid real literal, valid hexadecimal integer, valid tab escape, invalid escape sequence, invalid hexadecimal literal, and malformed real literal.
Expected result: lexical errors are reported for the malformed literals while the valid literals are still accepted correctly.
Observed output summary: this test was added to extend coverage beyond the supplied files and demonstrate independent verification of edge cases.

```
PS C:\Users\jamer\Desktop\CMSC 430 Compiler Theory and Design\Project 1\Project 1 Skeleton Code> cmd /c "type ""..\Project 1 Test Data\test8.txt"" | .\compile.exe"p\CMSC 430 Compiler Theory and Design\Project 1\Project 1 Skeleton Code>

   1  -- Punctuation symbols
   2
   3  ,:;() =>
   4
   5  // Valid identifiers
   6
   7  name_1
   8  name_1__a2_ab3
   9
  10  // Invalid identifiers
  11
  12  name___2
Lexical Error, Invalid Character _
Lexical Error, Invalid Character _
Lexical Error, Invalid Character _
  13  _name3
Lexical Error, Invalid Character _
  14  name4_
Lexical Error, Invalid Character _
  15
  16  // Integer Literals
  17
  18  23 #3aD
  19
  20  // Real Literals
  21
  22  123.45 .123 1.2E2 .1e+2 1.2E-2
  23
  24  // Character Literals
  25
  26  'A' '\n'
  27
  28  // Logical operators
  29
  30  & | !
  31
  31
  32  // Relational operators
  33
  34  = <> > >= < <=
  35
  36  // Arithmetic operators
  37
  38  + - * / % ^ ~
  39
  40  // Reserved words
  41
  42  begin case character else elsif end endcase endfold endif endswitch
  43  fold function if integer is left list of others real returns right
  44  switch then when
  45
Lexical Errors 5
Syntax Errors 0
Semantic Errors 0
```

3. Lessons Learned

One lesson from this project is that lexical rules need to be tested as a group instead of only one feature at a time. Identifier rules, operator rules, and literal rules can interact in subtle ways, especially when invalid input should still allow scanning to continue correctly.

Another lesson is that the listing and error-reporting code is just as important as the token rules. A scanner may recognize the correct tokens, but if it suppresses one of several errors on the same line or prints the wrong summary at the end, it still fails the project requirements.

I also learned that it is useful to keep the scanner logic structured around small, isolated categories such as identifiers, numbers, and character literals. That made it easier to compare behavior against the provided test cases and correct the exact invalid-identifier behavior required by test8.

4. Possible Improvements

If I were extending this project further, I would add more negative test cases for

malformed real literals, malformed hexadecimal literals, and invalid character escapes. I would also verify the Flex-based scanner generation directly in an environment where Flex is installed so the .l file can be built and tested end to end with the same automated checks.